



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SanDisk Hackathon

Track Name: AI / ML

Theme Name: Smart Storage Health Monitor

Problem Statement:

Modern storage devices such as SSDs and HDDs degrade over time due to wear, usage patterns, and environmental factors.

Unexpected failures can lead to **data loss, downtime, and operational disruptions**, especially in enterprise and cloud environments.

Current monitoring tools primarily provide **reactive insights** (e.g., SMART stats) but lack intelligent prediction and actionable recommendations.

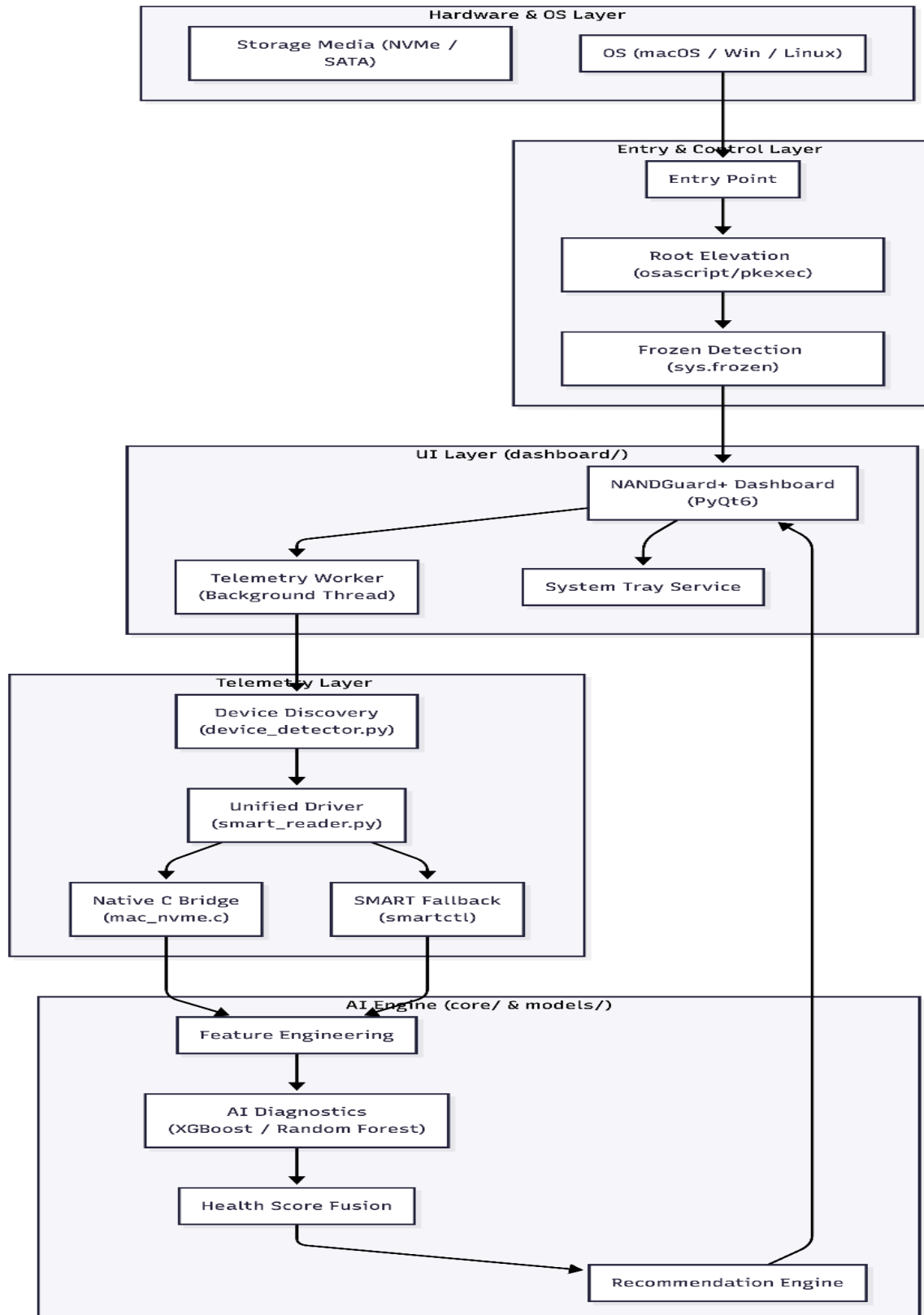
Therefore, there is a need for an **AI-driven system that continuously monitors storage telemetry, predicts failures, estimates remaining lifespan, and provides real-time health insights** to enable proactive maintenance.

Our Proposed Solution:

We propose **NANDGuard+**, an AI-driven, **cross-platform** storage health monitoring application built using PyQt6. The system continuously collects **SMART** telemetry from SSDs and HDDs, processes key indicators such as wear level, temperature, and error rates, and applies machine learning models to predict failures and estimate remaining useful life. It generates a unified health score and provides real-time alerts and actionable maintenance recommendations for proactive storage management.

Detailed Architecture/Block Diagram:

System architecture consists of the following layers:



Layer 0 — Hardware & OS Layer

Storage Media

NVMe (PCIe-based SSDs)

SATA SSDs

HDDs

Operating System

macOS

Windows

Linux

The OS exposes block devices and hardware telemetry interfaces accessible via system APIs and SMART utilities.

Layer 1 — Entry & Control Layer

Entry Point (main.py)

- Application bootstrap handler.
- Initializes runtime configuration.
- Detects OS type and environment.

Root Elevation

- macOS → osascript
- Linux → pkexec
- Windows → UAC prompt
- Ensures privileged access for SMART telemetry retrieval.

Frozen Detection (sys.frozen)

- Detects if application is packaged (PyInstaller/compiled build).
- Adjusts internal path handling for production deployment.
-

This layer ensures NANDGuard+ behaves like a proper installed driver utility.

Layer 2 — UI Layer (dashboard/ — PyQt6)

NANDGuard+ Dashboard (PyQt6)

- Professional desktop application interface.
- Cross-platform native styling.
- Driver-style monitoring utility design.

Telemetry Worker (Background Thread)

- Runs independently of UI thread.
- Polls telemetry every 30 seconds.
- Prevents UI freezing.
- Emits signals to update dashboard safely.

System Tray Service

- Runs in the background.
 - Displays notifications for:
 - Critical status
 - High temperature
 - Rapid wear
 - Allows quick open/exit functionality.
-
- **CLI Fallback Mode:** Terminal-based diagnostic output activated automatically in headless/server environments — full feature parity with the GUI.
 - **Alert & Recommendation Engine:** Translates ML outputs into plain-language maintenance actions (e.g., 'Backup immediately — Critical status detected').
 - **OS-Aware Orchestrator (main.py):** Detects the runtime environment and routes execution to the appropriate interface and telemetry backend.

Layer 3 — Telemetry Layer

1. Role: Device Discovery

- Enumerates all mounted storage devices using **psutil** across macOS, Windows, and Linux.
- Identifies device type (SSD / HDD / NVMe), model, and serial number.

2. Role: SMART Data Ingestion

- Invokes **smartmontools (smartctl)** to retrieve hardware-level SMART attributes.
- **Native Apple Telemetry (mac_nvme.c):**
 1. IOKit Integration: Communicates directly with the macOS I/O Registry using the IOKit framework to target **AppleANS3NVMeController** and **IONVMeBlockStorageDevice** services.
 2. Direct Memory Access: Attempts to retrieve the raw 512-byte NVMe SMART Log Page directly from the kernel-level "SMART Data" property, bypassing the need for external CLI parsing.
 3. Kernel Stats Fallback: If hardware SMART data is restricted, it queries the **IOBlockStorageDriver** for native Statistics (raw Bytes Read/Written), ensuring high-fidelity telemetry even under strict security policies.
- **Unified Orchestration (smart_reader.py):** Prioritizes the Native C Bridge for Apple Silicon, falls back to **smartctl -a** (handling macOS-specific warning codes), and triggers the Synthetic Simulator if no hardware is accessible.
- **Key attributes collected:** Power-On Hours, Wear Level Indicator, Reallocated Sectors, Uncorrectable Error Count, Media Errors, Temperature, Program/Erase (P/E) Cycles.
- Falls back to the Synthetic NAND Degradation Simulator when real hardware or **smartctl** is unavailable.

Frozen Detection (sys.frozen)

- Detects if the application is packaged (PyInstaller/compiled build).
- Adjusts internal path handling for production deployment.
- This layer ensures NANDGuard+ behaves like a proper installed driver utility.

Layer 4 — Data Processing Layer

- Data Cleaning: Handles missing, null, or out-of-range SMART attribute values with domain-informed imputation.
- Normalisation: Scales raw SMART values to a 0–1 range using min-max scaling calibrated against known healthy/failed drive datasets.

- Feature Engineering: Derives higher-order indicators — wear rate velocity (wear change per hour), error rate acceleration, temperature delta from a rolling baseline, and P/E cycle utilisation percentage.
- Composite Health Score: Computes a single 0–100 health score from a weighted combination of normalised attributes, providing an instant human-readable device condition summary.

Layer 5 — AI / ML Engine

Model	Task	Output
XGBoost Regressor	RUL Prediction	Estimated hours of useful life remaining (continuous value)
Random Forest Classifier	Health Classification	Healthy / Degrading / Critical + confidence score
Isolation Forest	Anomaly Detection	Anomaly flag + deviation score from normal telemetry baseline

- All models are pre-trained on the synthetic NAND degradation dataset and stored as serialised .pkl files in /models/. The pipeline is designed for drop-in retraining as real device data becomes available.

Detailed Methodology:

Device Discovery & Data Acquisition

- The system detects available storage devices and collects hardware telemetry using SMART monitoring tools.

Feature Engineering

- Raw attributes are processed to derive meaningful indicators such as wear rate, error trends, and utilization metrics.

Health Scoring Mechanism

- The overall storage health is represented using a **composite health score (0–100)** that integrates multiple predictive and telemetry indicators into a single interpretable metric.

$$HealthScore = 0.4 \times Normalized_RUL + 0.3 \times (1 - Failure_Probability) + 0.2 \times (1 - Wear_Percentage) + 0.1 \times (1 - Anomaly_Score)$$

Normalized RUL: Predicted remaining lifespan of the storage device obtained from the XGBoost regression model and scaled between 0–1.

Failure Probability: Likelihood of the device being in a degrading or critical state derived from the Random Forest classification model output.

Wear Percentage: Proportion of physical NAND endurance consumed, calculated from SMART telemetry attributes such as wear leveling count or percentage used.

Anomaly Score: Degree of abnormal behavior in device telemetry detected using the Isolation Forest anomaly detection model.

Machine Learning Analysis

- Classification model determines current health status
- Regression model predicts remaining useful life
- Anomaly detection identifies unusual behavior patterns

Visualization & Reporting

Insights are presented through a real-time dashboard with alerts and recommendations.

Simulation Mode (if hardware unavailable)

Synthetic data generator simulates NAND degradation trends to demonstrate predictive capability.

Deliverables Expected:

- A Cross Platform working prototype of NANDGuard+ Driver application
- Real-time health monitoring dashboard
- ML models for prediction and anomaly detection
- Synthetic dataset and simulation module
- Documentation (architecture, methodology, and usage)
- Demonstration of predictive insights and alerts

TEAM: FSEM